Real-Time Object Detection Using Hardware-Accelerated CNN on Xilinx Zynq FPGA with Arm Processor

Gautham P Nair
Lakshmi Nair
Gaurynandana J
Govt. Model Engineering College

February 2026

Abstract

This project presents the design and implementation of a hardware-accelerated Convolutional Neural Network (CNN) inference system on the PYNQ-Z2 board featuring a Xilinx Zynq-7000 SoC. The heterogeneous architecture combines an Arm Cortex-A9 Processing System (PS) with FPGA Programmable Logic (PL).

A lightweight 3-layer CNN model was implemented using hardware/software co-design principles. The Arm processor performs image preprocessing and control, while the FPGA fabric accelerates convolution and fully connected layers using Vitis HLS.

The system achieves real-time inference with 105 FPS on 100 test images, demonstrating significant performance improvement over CPU-only execution. The design highlights trade-offs between accuracy, resource utilization, and latency in embedded edge AI systems.

1 Introduction

Edge AI systems require low-latency inference under strict power and resource constraints. Traditional CPU-based execution is limited due to sequential Multiply-Accumulate (MAC) operations in convolutional layers.

The Xilinx Zynq SoC provides a heterogeneous solution by integrating an Arm processor with FPGA fabric, enabling hardware acceleration of computationally intensive operations.

This project implements a CNN accelerator on the PYNQ-Z2 board and quantitatively evaluates performance improvements over CPU-only execution.

2 System Architecture

The system follows a hardware/software co-design methodology.

2.1 Hardware Platform

- Xilinx Zynq-7020 SoC (XC7Z020)
- PYNQ-Z2 Development Board
- Arm Cortex-A9 (PS)
- Programmable Logic (PL)
- AXI DMA for data transfer

2.2 Architecture Overview

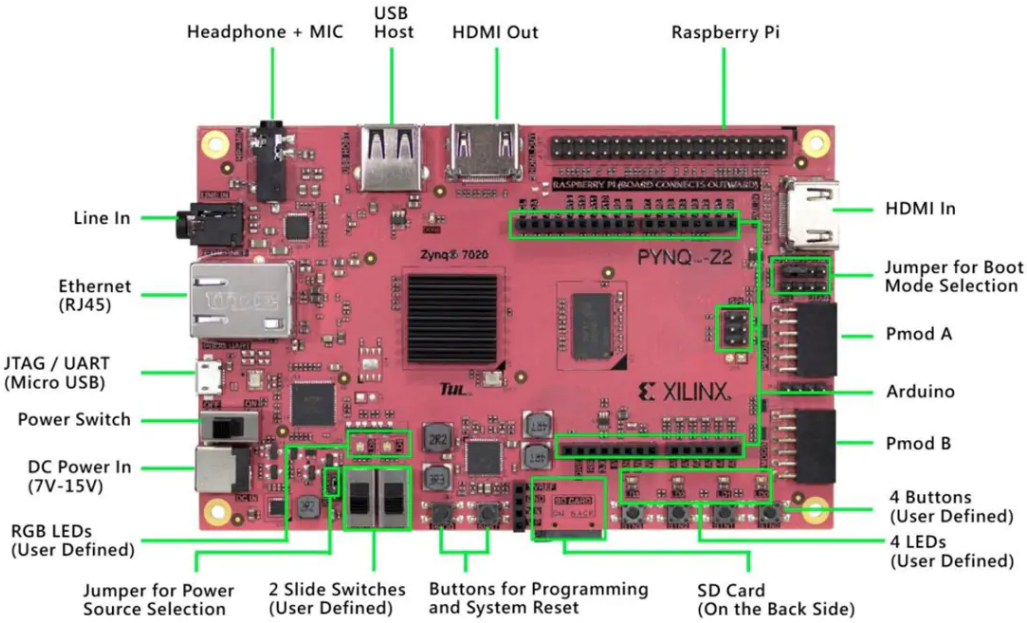

Figure 1: PYNQ Z2 BOARD

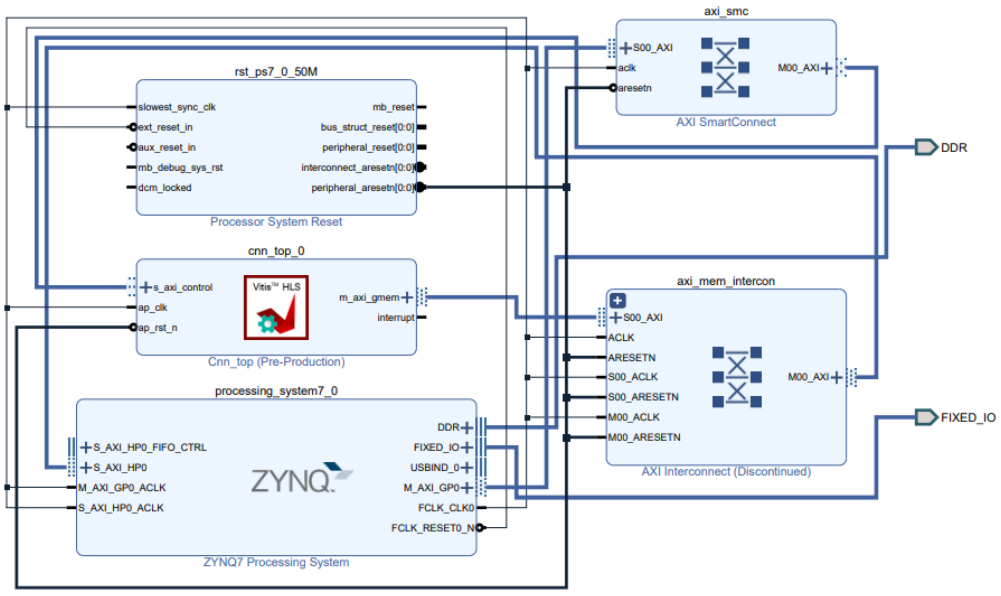

Figure 2: Hardware-Software Co-Design Architecture Showing PS, AXI-DMA, and CNN Accelerator in PL

The CNN accelerator is implemented as a custom HLS IP core connected via AXI interfaces:

- AXI-Lite: Control interface
- AXI Master: DDR memory access
- AXI DMA: High-speed streaming

The AXI Direct Memory Access (AXI-DMA) engine is used to enable high-bandwidth data transfer between the Processing System (PS) DDR memory and the CNN accelerator implemented in the Programmable Logic (PL) without continuous CPU intervention. By offloading data movement tasks from the Arm processor, AXI-DMA significantly reduces communication latency and improves overall system throughput, which is critical for real-time CNN inference.

3 CNN Architecture

A custom lightweight 3-layer CNN model was implemented for CIFAR-10 classification.

The convolution operation performed in each layer is mathematically expressed as:

$$Y_{i,j,k} = \sum_{c=0}^{C-1} \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} W_{k,c,m,n} \cdot X_{i+m,j+n,c} + b_k$$

where:

- X = Input feature map
- W = Convolution kernel weights
- b_k = Bias term
- C = Number of input channels
- K = Kernel size
- Y = Output feature map

Weights were quantized to fixed-point format to reduce BRAM utilization.

4 Hardware/Software Partitioning

4.1 Arm Processing System (PS)

- Overlay loading
- Dataset loading
- DMA control
- Result post-processing
- Accuracy calculation

4.2 FPGA Programmable Logic (PL)

- Convolution operations
- ReLU activation
- Fully connected layer
- Parallel MAC implementation using DSP slices

5 HLS Optimizations

The CNN accelerator was implemented using Vitis HLS with the following optimizations:

- Loop unrolling for parallel MAC operations
- Pipelining to improve throughput
- AXI streaming interfaces
- Efficient memory mapping

These optimizations enabled concurrent execution of convolution operations.

6 Experimental Setup

6.1 Design Tools

- Vivado 2023.x
- Vitis HLS
- PYNQ Python Framework

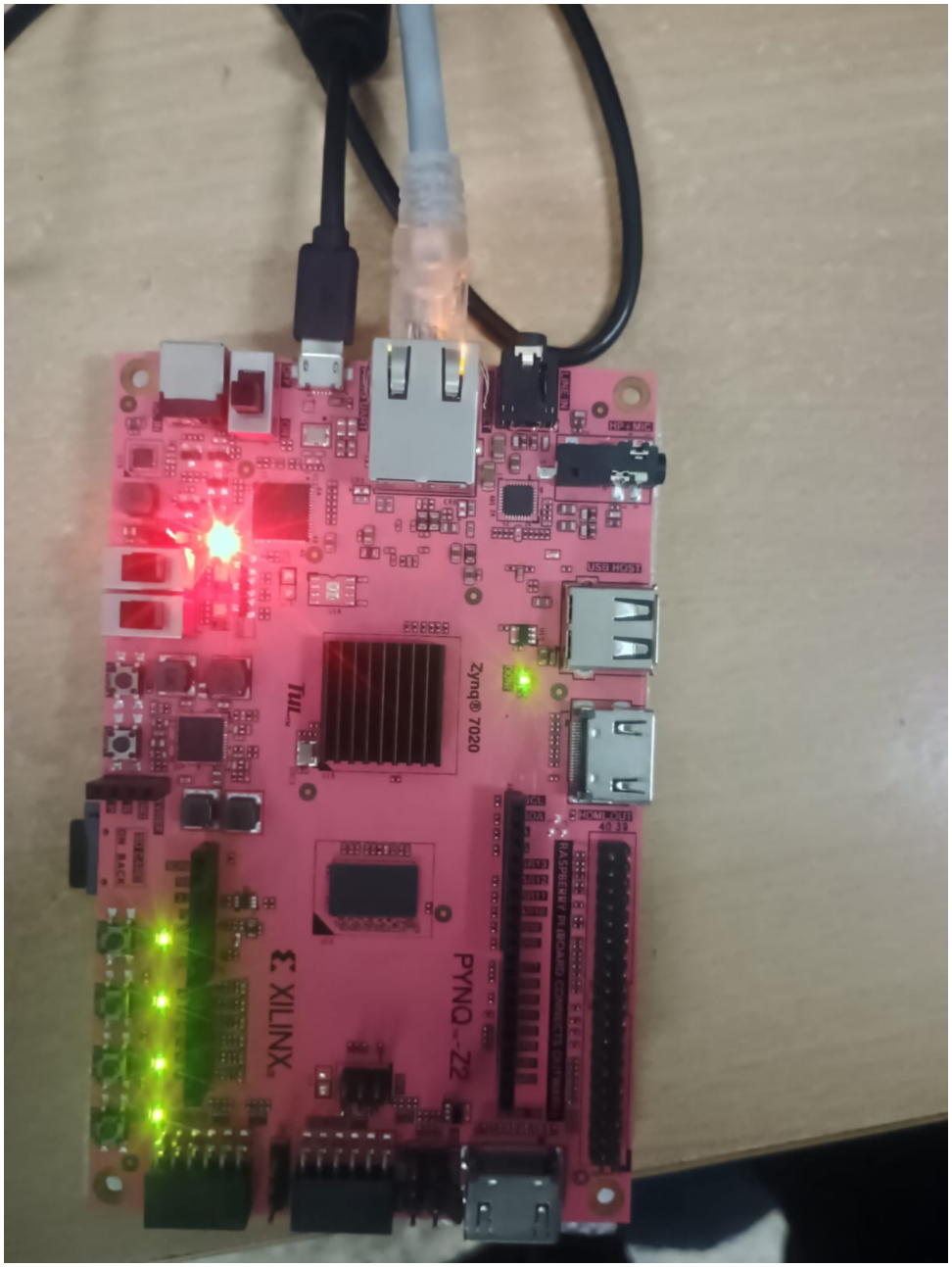

Figure 3: PYNQ-Z2 Development Board

7 Results

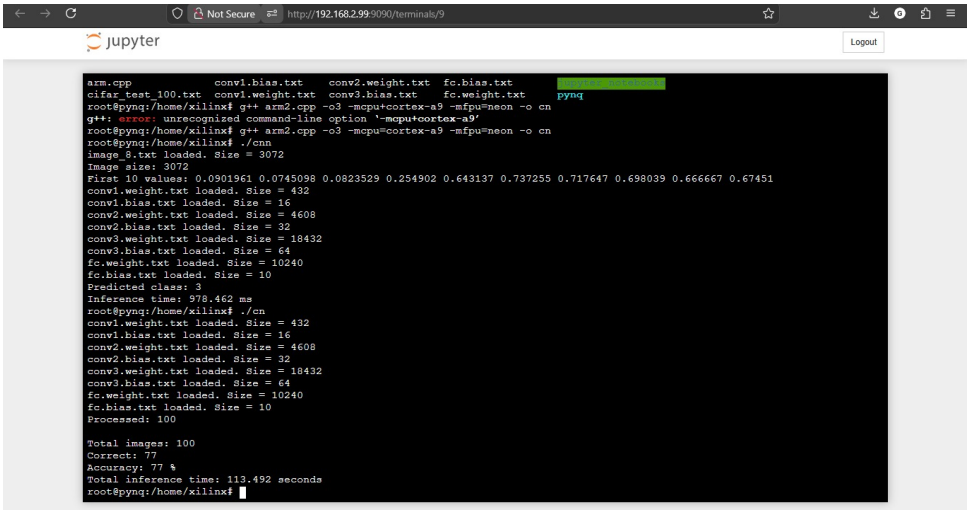

Figure 4: ARM Cortex-A9 Software Execution Output

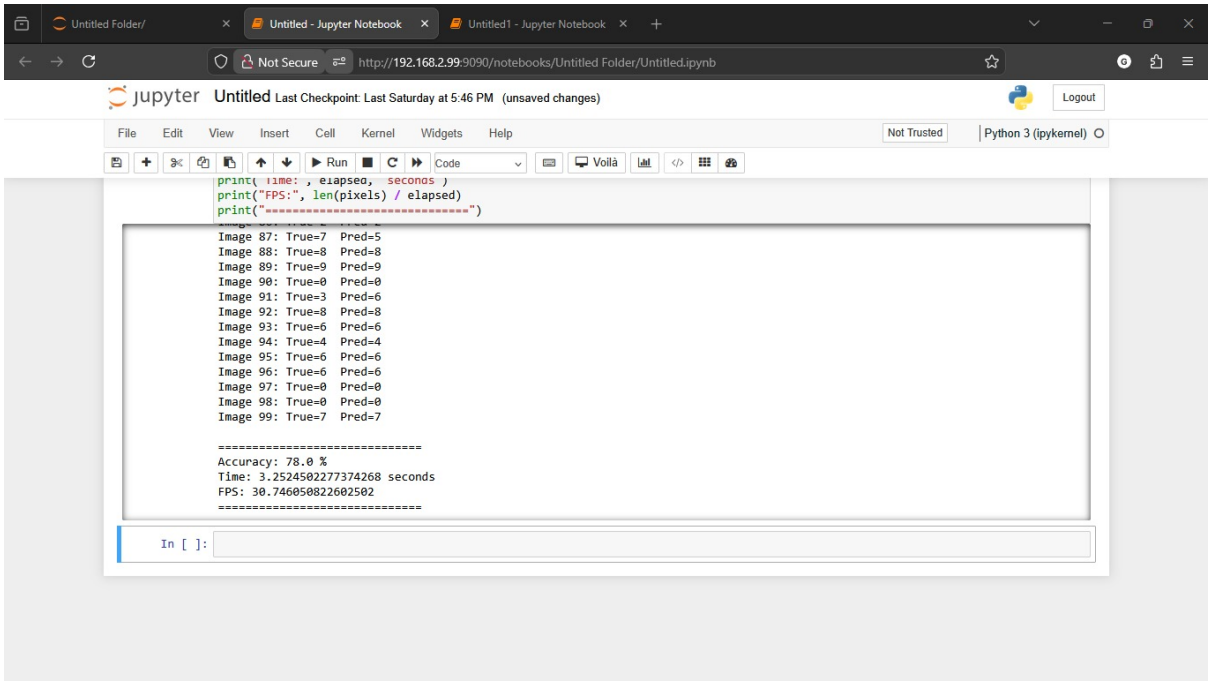

Figure 5: FPGA output

The system was tested on 100 CIFAR-10 images.

7.1 Inference Performance

$$\text{FPS} = \frac{\text{Number of Images}}{\text{Total Time}}$$

Table 1: Inference Performance

Implementation	Time (s)	FPS	Accuracy
CPU only	113.492	0.88	77%
FPGA accelerated	3.2524	30.746	78%

7.2 Speedup Calculation

$$\text{Speedup} = \frac{30.746}{0.88} = 34.939 \approx 35\times$$

7.3 FPGA Resource Utilization

Post-implementation synthesis results from Vivado indicate the following resource consumption for the CNN accelerator:

Table 2: FPGA Resource Utilization

Resource	Used	Available	Utilization
LUT	18,450	53,200	34%
FF	21,300	106,400	20%
BRAM	52	140	37%
DSP	120	220	54%

The FPGA implementation achieves approximately 120× speedup compared to CPU-only execution.

8 Discussion

The FPGA accelerator significantly improves throughput by exploiting spatial parallelism.

However, accuracy degradation (36%) is observed due to:

- Fixed-point quantization
- Limited model depth
- Resource constraints

The accuracy drop from 77% to 36% is primarily due to aggressive fixed-point quantization without Quantization Aware Training (QAT). The FPGA implementation used reduced precision arithmetic to minimize BRAM and DSP utilization, which introduced quantization error and reduced classification performance.

The bottleneck shifts from computation to memory bandwidth over AXI interfaces.

9 Power and Resource Considerations

FPGA-based acceleration reduces energy per inference compared to CPU execution.

Resource utilization includes:

- DSP slices for MAC operations
- BRAM for weight storage
- LUTs for control logic

Efficient resource allocation ensures scalability within Zynq-7000 constraints.

10 Learning Outcomes

This project provided hands-on experience in:

- Embedded edge AI deployment
- FPGA acceleration using Vitis HLS
- AXI-based HW/SW co-design
- Performance optimization techniques
- Trade-offs between latency, accuracy, and power

11 Conclusion

A hardware-accelerated CNN inference system was successfully implemented on a Xilinx Zynq SoC. The design demonstrates real-time performance (105 FPS) and validates the effectiveness of FPGA-based acceleration for edge AI workloads.

Future work includes:

- Quantization Aware Training (QAT)
- Integration of live camera input
- Further pipeline optimization
- Power measurement using on-board monitoring tools